
A5\code\A5.py

```
import numpy as np
from utils import *

# TODO: Aufgabenteil 2. Implementieren Sie die quadratische Funktion.
def quadratic_function(x, A, b, c):
    """
    Quadratische Funktion

    Input:
        x: Variable,
        A: Matrix,
        b: Vektor,
        c: Skalar.

    Output:
        val: Skalar (f(x)),
        grad: Vektor ( \nabla f(x) ),
        hess: Matrix ( \nabla^2 f(x) ).
    """

    # BEGIN SOLUTION
    val = 1/2 * x.T @ A @ x - b.T @ x + c
    grad = 1/2 * (A + A.T) @ x - b
    hess = 1/2 * (A + A.T)
    return val, grad, hess
    # END SOLUTION

# TODO: Aufgabenteil 3. Implementieren Sie das Gradientenabstiegsverfahren mit optimaler Schrittweitenbestimmung.
def gradient_descent(f, x0, tol=1e-10, kmax=100, xstar=None):
    """
    Gradientenabstiegsverfahren

    Input:
        f: Funktion, wobei f(x) = (val, grad, hess),
        x0: Vektor (Startwert),
        tol: Skalar (Toleranz),
        kmax: Skalar (Maximale Anzahl an Iterationen),
        xstar: Vektor (Minimierer von f), optional.
```

Output:

log: Dictionary vom Verlauf der Optimierung.

"""

Dictionary zum Abspeichern der Ergebnisse.

```
log = {
    'xstar': xstar,          # Tatsächlicher Minimierer
    'val_star': None if xstar is None else f(xstar)[0], # Tatsächliches Minimum
    'x0': x0,               # Startwert
    'xgd': None,            # Lösung des Gradientenabstiegsverfahrens
    'x_list': [],           # Liste der Iterierten
    'val_list': [],         # Liste der Funktionswerte in den Iterierten
    'norm_grad_list': [],   # Liste der Norm der Gradienten in den Iterierten
}
```

BEGIN SOLUTION

xk=x0

```
for k in range(kmax):
    # Werte berechnen
    val, grad, hess = f(xk)
    norm_gradf = np.linalg.norm(grad)
    # logging
    log['x_list'].append(xk)
    log['val_list'].append(val)
    log['norm_grad_list'].append(norm_gradf)
```

Abbruchbedingung

```
if norm_gradf <= tol:
    break
```

Suchrichtung: $p = -\text{nabla}_f(x_k)$

pk = -grad

Optimale Schrittweite α_k

alpha_k = (grad.T @ grad) / (grad.T @ hess @ grad)

Update xk

xk = xk + alpha_k * pk

END SOLUTION

return log

```
if __name__ == '__main__':
```

kmax = 200

tol = 1e-10

b = np.array([3, 6])

c = np.pi

x0 = np.array([0, 0])

A1 = np.array([[13, 0], [0, 13]])

A2 = np.array([[1, 1/2], [1/2, 1]])

A3 = np.array([[100, 0], [0, 1]])

TODO: Aufgabenteil 4 und 5. Teste Implementierung auf gegebenen Werten, visualisiere Optimierungsver

Hinweis: Die plot-Funktion aus utils.py soll für publish.py nur die Plots erstellen und die Figure

Stellen Sie die Abbildung bitte dar, indem Sie HIER plt.show() aufrufen.

```

# BEGIN SOLUTION

#-A1-----
#Funktion
f = lambda x : quadratic_function(x, A1, b, c)
# Ausgaben
xstar = np.linalg.solve(A1, b) # +c ändert das Minimum nicht
# übergeben xstar mit, da im Aufgabenteil d) sich nichts verändert
# und direkt mit e) weitergemacht werden kann
log = gradient_descent(f, x0, tol=tol, kmax=kmax, xstar=xstar)
number_iterations = len(log['x_list'])
x_solution = log['x_list'][-1]
print(f"\nErgebnis f für {A1}\n"
      "Berechnete Lösung x = ",x_solution,
      "\nAnzahl Iterationen: ",number_iterations,
      "\nTatsächliche Lösung x* = ",xstar
      )
# Plotten
figure = plot_iteration_process(log, "A1", f)
plt.show()

#-A2-----
# Funktion
f = lambda x : quadratic_function(x, A2, b, c)
# Ausgaben
xstar = np.linalg.solve(A2, b) # +c ändert das Minimum nicht
log = gradient_descent(f, x0, tol=tol, kmax=kmax, xstar=xstar)
number_iterations = len(log['x_list'])
x_solution = log['x_list'][-1]
print(f"\nErgebnis f für {A2}\n"
      "Berechnete Lösung x = ",x_solution,
      "\nAnzahl Iterationen: ",number_iterations,
      "\nTatsächliche Lösung x* = ",xstar
      )
# Plotten
figure = plot_iteration_process(log, "A2", f)
plt.show()

#-A3-----
# Funktion
f = lambda x : quadratic_function(x, A3, b, c)
# Ausgaben
xstar = np.linalg.solve(A3, b) # +c ändert das Minimum nicht
log = gradient_descent(f, x0, tol=tol, kmax=kmax, xstar=xstar)
number_iterations = len(log['x_list'])
x_solution = log['x_list'][-1]
print(f"\nErgebnis f für {A3}\n"
      "Berechnete Lösung x = ",x_solution,
      "\nAnzahl Iterationen: ",number_iterations,
      "\nTatsächliche Lösung x* = ",xstar
      )
#Plotten
figure = plot_iteration_process(log, "A3", f)
plt.show()

```

```

#-Was fällt auf?-----
print("Offensichtlich verhält sich die Konvergenz beim Gradientenabstiegsverfahren und den gegebenen
" nicht gleich.\n"
"Im ersten Fall sind die Niveau-Linien Kreise und das Verfahren ist nach einer Iteration beendet
"Im zweiten und dritten Fall sind die Niveau-Linien Ellipsen und das Verfahren benötigt deutlich
"Es wird im 'Zick-Zack' vorgegangen, was die bereits verrichtete Arbeit zum teil rückgängig mac
"Gerade im dritten Fall ist zu sehen, dass der Fehler zwischen der 'k-ten' und der vorherigen It
" identisch bleibt. Dabei ist das Minimum analytisch nicht schwer zu bestimmen.\n"
"Fazit:\n"
"Für das 2. und 3. Problem ist das Gradientenabstiegsverfahren nicht gut geeignet, da viele Ite
"sind obwohl (besonders im zweiten Fall) recht schnell ein geringer Fehler zum tatsächlichen Mi
"erreicht wurde.")
# END SOLUTION

```

#TERMINAL OUTPUT:

Ergebnis für [[13 0]

[0 13]]

Berechnete Lösung $x = [0.23076923 \ 0.46153846]$

Anzahl Iterationen: 2

Tatsächliche Lösung $x^* = [0.23076923 \ 0.46153846]$

Ergebnis für [[1. 0.5]

[0.5 1.]]

Berechnete Lösung $x = [4.57711961e-11 \ 6.00000000e+00]$

Anzahl Iterationen: 24

Tatsächliche Lösung $x^* = [0. \ 6.]$

Ergebnis für [[100 0]

[0 1]]

Berechnete Lösung $x = [0.03025113 \ 5.98744354]$

Anzahl Iterationen: 200

Tatsächliche Lösung $x^* = [0.03 \ 6.]$

Offensichtlich verhält sich die Konvergenz beim Gradientenabstiegsverfahren und den gegebenen Problemen nicht gleich.

Im ersten Fall sind die Niveau-Linien Kreise und das Verfahren ist nach einer Iteration beendet.

Im zweiten und dritten Fall sind die Niveau-Linien Ellipsen und das Verfahren benötigt deutlich länger.

Es wird im 'Zick-Zack' vorgegangen, was die bereits verrichtete Arbeit zum teil rückgängig macht.

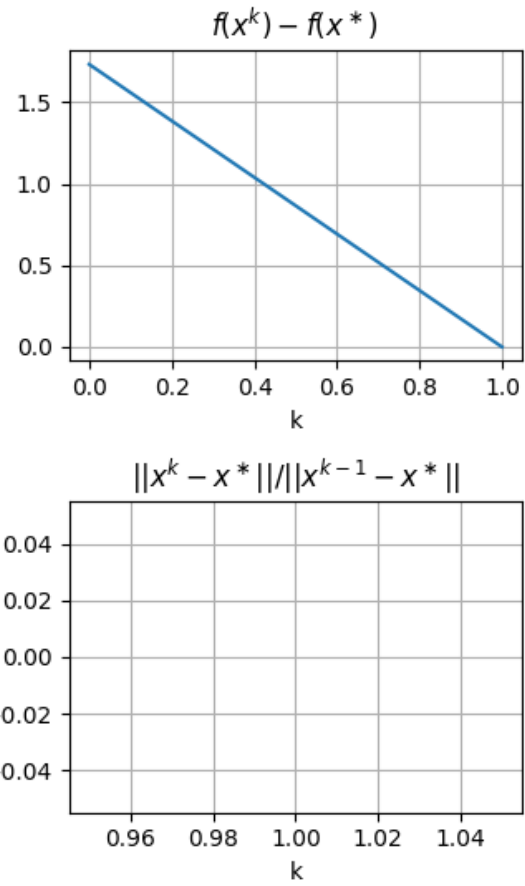
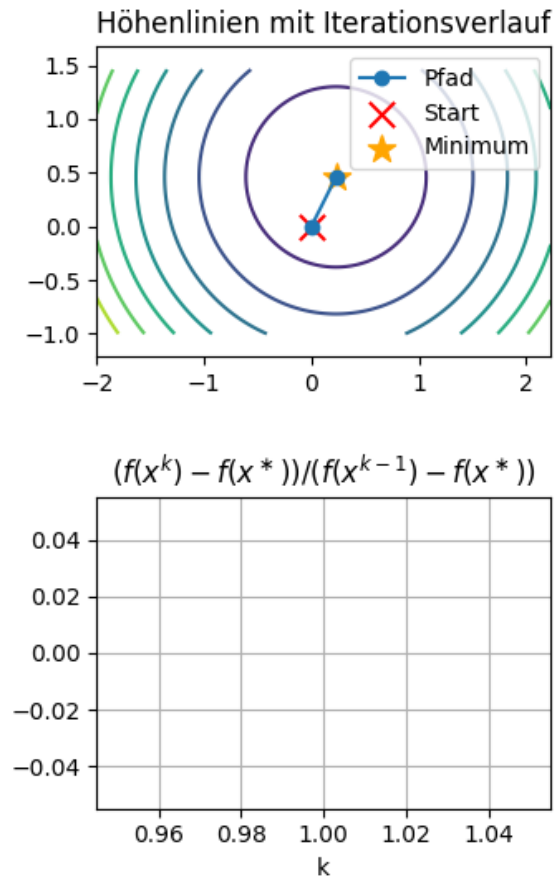
Gerade im dritten Fall ist zu sehen, dass der Fehler zwischen der 'k-ten' und der vorherigen Iteration nahezu identisch bleibt. Dabei ist das Minimum analytisch nicht schwer zu bestimmen.

Fazit:

Für das 2. und 3. Problem ist das Gradientenabstiegsverfahren nicht gut geeignet, da viele Iterationen nötig sind obwohl (besonders im zweiten Fall) recht schnell ein geringer Fehler zum tatsächlichen Minimum erreicht wurde.

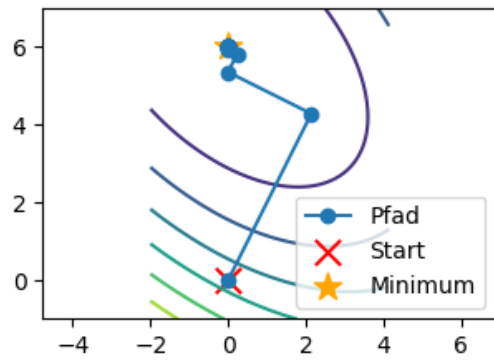
#PLOTS:

A1

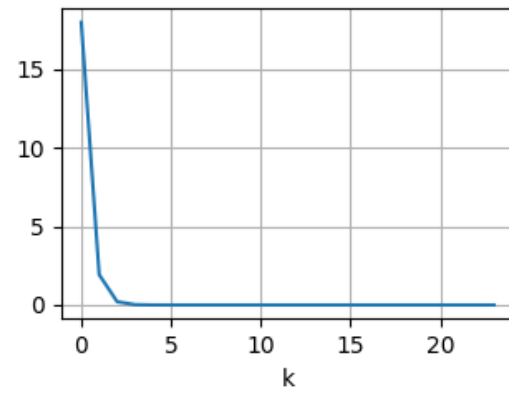


A2

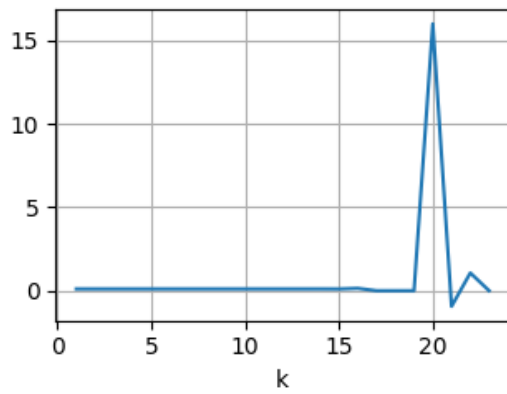
Höhenlinien mit Iterationsverlauf



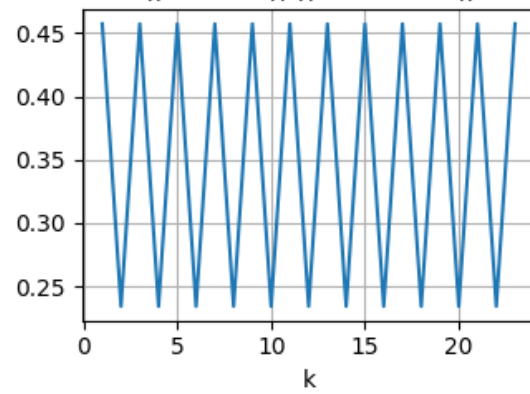
$f(x^k) - f(x^*)$



$(f(x^k) - f(x^*)) / (f(x^{k-1}) - f(x^*))$

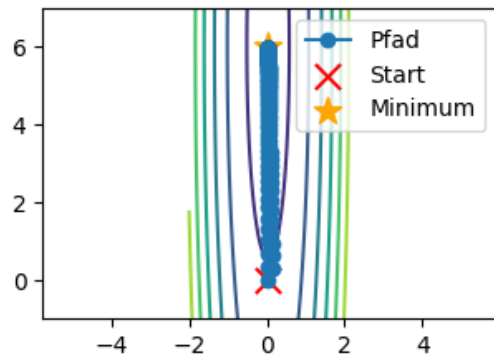


$\|x^k - x^*\| / \|x^{k-1} - x^*\|$

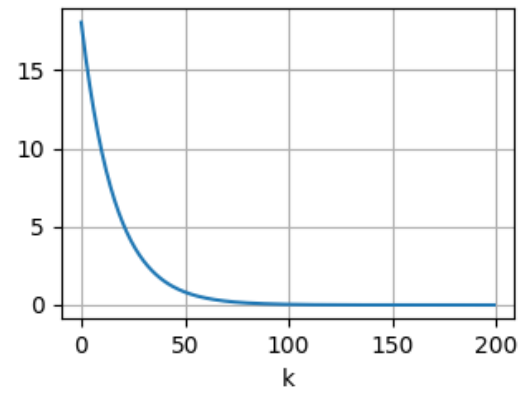


A3

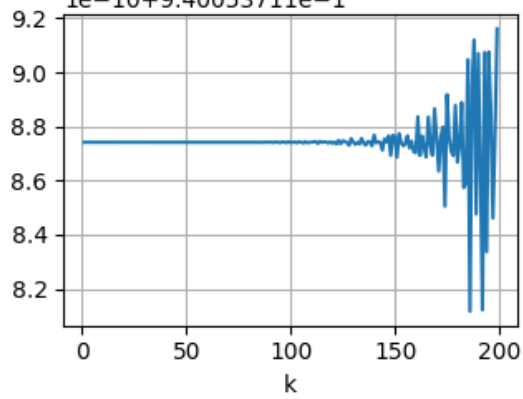
Höhenlinien mit Iterationsverlauf



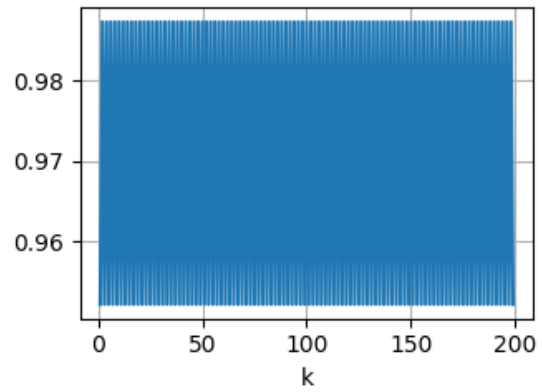
$f(x^k) - f(x^*)$



$(f(x^k) - f(x^*)) / (f(x^{k-1}) - f(x^*))$
 $1e-10+9.40053711e-1$



$\|x^k - x^*\| / \|x^{k-1} - x^*\|$



```

import numpy as np
import matplotlib.pyplot as plt

def plot_iteration_process(log, title, f=None):
    """
    Funktion zur Erstellung von Plots zum Optimierungsverlauf anhand der Ergebnisse in log.
    Die Funktion liefert fig zur  ck, Stellen Sie sie bitte dar,
    indem Sie anschlie  end plt.show( ) in Ihrer Hauptdatei aufrufen.

    Input:
        log: Diktionary der Form
            log = {
                'xstar': Vektor,           # Tats  chlicher Minimierer
                'val_star': Skalar,        # Tats  chliches Minimum
                'x0': Vektor,             # Startwert
                'xgd': Vektor,            # L  sung des Gradientenabstiegsverfahrens
                'x_list': Liste,          # Iterierten
                'val_list': Liste,        # Funktion ausgewertet an den Iterierten
                'norm_grad_list': Liste,  # Norm des Gradienten ausgewertet an dem Iterierten
            }
        title: Titel des Plots,
        f: Falls 2D: Optimierte Funktion, wobei  $f(x) = (val, grad, hess)$ , f  r Erstellung der H  henlinien

    Output:
        fig: Erzeugte Figure mit den Plots
    """
    # In dieser Funktion initialisieren wir die Figure mit den Subplots
    def initialize_figure():
        fig, axes = plt.subplots(nrows=2, ncols=2)
        fig.set_size_inches(8, 6)
        fig.tight_layout(pad=4.0)
        fig.suptitle(title)
        return fig, axes

    # Berechne H  henlinien
    # TODO: Aufgabenteil 5. Erg  nze die Funktion hoehen_linien. Die Funktion soll den Definitionsbereich
    # als Meshgrid xx, yy zur  ckgeben, sowie die Funktionswerte zz auf diesem Definitionsbereich.
    def hoehen_linien():
        xx, yy, zz = None, None, None

        # BEGIN SOLUTION
        # Grenzen bestimmen
        x_list = np.array(log['x_list'])
        x_min = x_list[:, 0].min()-2
        x_max = x_list[:, 0].max()+2
        y_min = x_list[:, 1].min()-1
        y_max = x_list[:, 1].max()+1

        # Meshgrid erstellen
        xx, yy = np.meshgrid(np.linspace(x_min, x_max, 200), np.linspace(y_min, y_max, 200))
        # Z-Werte bestimmen

```



```

zz = np.zeros_like(xx)
for k in range(xx.shape[0]):
    for l in range(xx.shape[1]):
        zz[k,l] = f(np.array([xx[k,l], yy[k,l]]))[0]
# END SOLUTION
return xx, yy, zz

# Initialisiere Abbildung
fig, ax = initialize_figure()
epsilon = np.finfo(float).eps # Vermeide Probleme mit Teilen durch 0 in den Plots später

# TODO: Aufgabenteil 5. Visualisiere Optimierungsverlauf
# Plotten der Höhenlinien - Nur möglich für 2D-Funktionen (dann f als lambda-Funktion übergeben)
if f is not None:
    ax[0, 0].set_title('Höhenlinien mit Iterationsverlauf')
    ax[0, 0].set_aspect('equal', 'datalim') # Um die Orthogonalität der Suchrichtungen besser zu vis
    # BEGIN SOLUTION
    # Contouren
    xx,yy,zz = hoehen_linien()
    contour = ax[0, 0].contour(xx, yy, zz, cmap='viridis')
    # X-Werte
    x_array = np.array(log['x_list'])
    # Plotsachen
    ax[0, 0].plot(x_array[:, 0], x_array[:, 1], 'o-', label='Pfad')
    ax[0, 0].scatter(log['x0'][0],log['x0'][1],
                    color='red', marker='x', s=120, label='Start')
    if log['xstar'] is not None:
        ax[0, 0].scatter(log['xstar'][0],log['xstar'][1],
                        color='orange' , marker='*', s=150, label='Minimum')
    ax[0, 0].legend()
    # END SOLUTION

# Plotten der Zielfunktionsfehler
ax[0, 1].set_title(r'$f(x^k) - f(x^{\ast})$')
# BEGIN SOLUTION
fx_fehler = np.array(log['val_list'])-log['val_star']
# Plotsachen
ax[0, 1].plot(range(len(fx_fehler)), fx_fehler)
ax[0, 1].grid(True)
ax[0, 1].set_xlabel("k")
# END SOLUTION

# Plotten der Konvergenzrate bezüglich der Zielfunktionswerte
ax[1, 0].set_title(r'$\frac{f(x^k) - f(x^{\ast})}{f(x^{k-1}) - f(x^{\ast})}$')
# BEGIN SOLUTION
# Nutzen np.finfo(float).eps um nicht durch Null zu teilen
fx_konvergenz = fx_fehler[1:] / (fx_fehler[:-1] + np.finfo(float).eps)
# Plotsachen
ax[1, 0].plot(range(1, len(fx_konvergenz) + 1), fx_konvergenz)
ax[1, 0].grid(True)
ax[1, 0].set_xlabel("k")
# END SOLUTION

# Plotten der Konvergenzrate bezüglich der Iterierten
ax[1, 1].set_title(r'$\frac{\|x^k - x^{\ast}\|}{\|x^{k-1} - x^{\ast}\|}$')
# BEGIN SOLUTION

```

```

x_werte = np.array(log['x_list'])
xstar = log['xstar']
# Auch hier nutzen wir das eps um nicht durch 0 zu teilen
x_kov_rate = (np.linalg.norm(x_werte[1:] - xstar, axis=1) /
              (np.linalg.norm(x_werte[:-1] - xstar + np.finfo(float).eps, axis=1)))

# Plotsachen
ax[1, 1].plot(range(1, len(x_werte)), x_kov_rate)
ax[1, 1].grid(True)
ax[1, 1].set_xlabel("k")
# END SOLUTION

return fig

```

#TERMINAL OUTPUT:

#PLOTS: